TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

LẬP TRÌNH CHO THIẾT BỊ DI ĐỘNG

THÁI NGUYÊN, THÁNG 5 NĂM 2024

Chủ đề 1: Tổng quan về lập trình di động

Nội dung chính:

Giới thiệu:
Lập trình di động phát triển ứng dụng trên các thiết bị như điện thoại, máy tính bảng, hỗ trợ kết nối và nâng cao chất lượng cuộc sống.

Tầm quan trọng:

- Công cụ thiết yếu trong đời sống hiện đại.
- Tạo cơ hội lớn trong kinh doanh và dịch vụ.

Mục tiêu bài học:

- Hiểu các phương pháp lập trình di động.
- Làm quen với công cụ và ngôn ngữ Kotlin.
- Thực hành cơ bản với Android Studio.

Ví dụ:
Ứng dụng Grab hỗ trợ giao thông, giao đồ ăn, thanh toán trực tuyến.

Tổng quan về lập trình di động

Lập trình di động tập trung phát triển ứng dụng chạy trên các thiết bị di động.

Ngày càng phổ biến nhờ số lượng thiết bị di động vượt quá 5 tỷ trên toàn cầu.

Ứng dụng di động thay đổi cách con người giao tiếp, làm việc và giải trí.

Cung cấp giải pháp cho nhiều ngành: y tế, giáo dục, thương mại, giải trí.

Ví dụ: Ứng dụng Grab giúp kết nối tài xế và khách hàng trong thời gian thực.

Tại sao lập trình di động quan trọng?

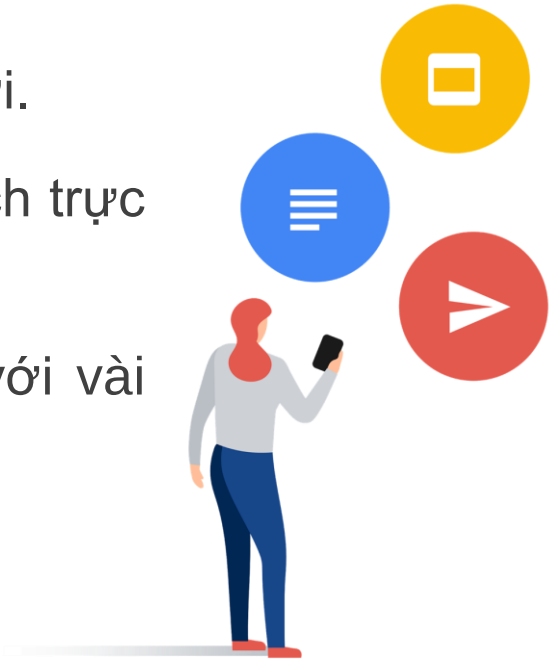
Thiết bị di động đã trở thành một phần không thể thiếu trong cuộc sống.

Các ứng dụng di động tạo ra cơ hội lớn trong các ngành công nghiệp chưa số hóa.

Kết nối người dùng với thông tin và dịch vụ nhanh chóng, tiện lợi.

Mang lại giá trị kinh tế lớn qua doanh thu quảng cáo và giao dịch trực tuyến.

Ví dụ: Zalo Pay giúp thực hiện các giao dịch thanh toán chỉ với vài thao tác.



Các đặc điểm lập trình di động

Tính đa dạng thiết bị: kích thước, hệ điều hành và hiệu suất khác nhau.

Hạn chế tài nguyên: Cần quản lý bộ nhớ, CPU, và năng lượng hiệu quả.

Tương tác người dùng: Thiết kế giao diện tối ưu cho cảm ứng.

Kết nối mạng không ổn định: Phải đảm bảo xử lý offline hiệu quả.

Bảo mật: Đảm bảo an toàn thông tin người dùng trước các tấn công mạng.

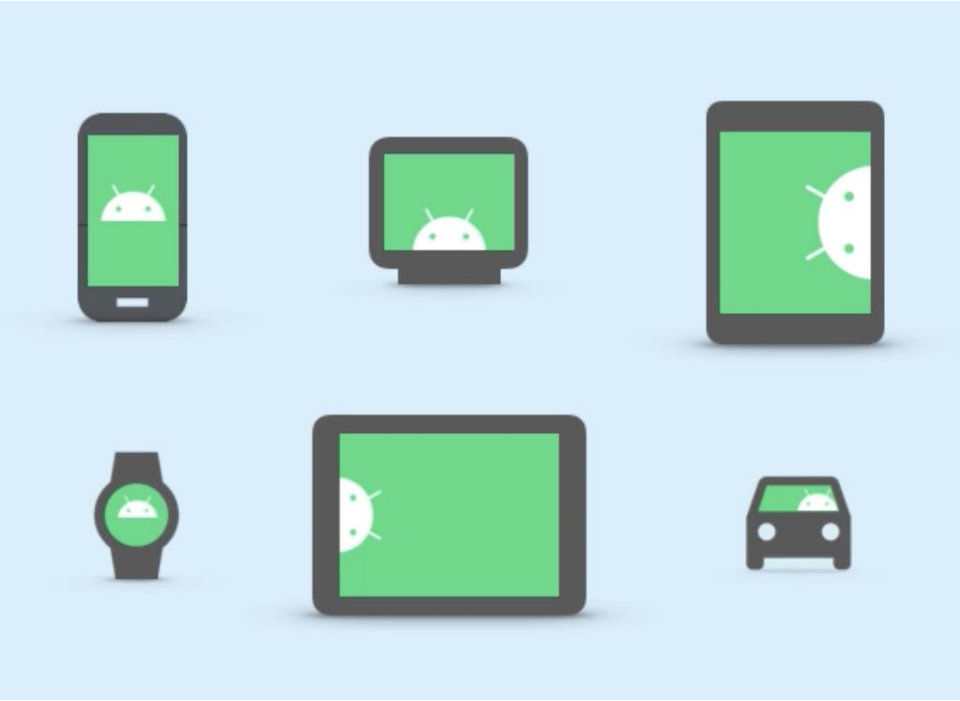
Tính đa dạng thiết bị

Hàng trăm loại thiết bị khác nhau chạy Android, từ điện thoại đến đồng hồ thông minh.

Các yếu tố: kích thước màn hình, độ phân giải, tỷ lệ khung hình khác biệt.

Ứng dụng cần hỗ trợ nhiều phiên bản hệ điều hành và phần cứng.

Giải pháp: Sử dụng các công cụ như ConstraintLayout, Jetpack Compose.



Tối ưu hóa tài nguyên

- Bộ nhớ hạn chế: Phải quản lý chặt chẽ việc phân bổ và giải phóng bộ nhớ.
- Tiêu hao pin: Sử dụng API tiết kiệm năng lượng và tránh thao tác không cần thiết.
- Hiệu suất CPU: Ưu tiên thuật toán nhẹ nhàng, giảm thiểu công việc nền.
- Giải pháp: Dùng RecyclerView để tối ưu danh sách hiển thị lớn.
- Ví dụ: Các ứng dụng như Facebook tải nội dung dần (lazy loading).

Giao diện cảm ứng và trải nghiệm người dùng

Thiết kế giao diện tối ưu với các cử chỉ như chạm, vuốt, kéo thả.

Đảm bảo giao diện phản hồi nhanh chóng để giữ người dùng hài lòng.

Hỗ trợ đa ngôn ngữ và đa văn hóa trong thiết kế.

Áp dụng các nguyên tắc Material Design để tăng tính trực quan và nhất quán.

Ví dụ: Giao diện của Instagram tối ưu hóa cho chạm và vuốt để duyệt ảnh.

Phương pháp phát triển gốc (Native Development)

Phát triển gốc nghĩa là viết mã riêng cho từng nền tảng: Android hoặc iOS.

Sử dụng ngôn ngữ gốc: Kotlin/Java cho Android, Swift/Objective-C cho iOS.

Tận dụng đầy đủ API và tài nguyên hệ thống.

Ứng dụng gốc cho phép truy cập trực tiếp vào cảm biến, camera và GPS.

Ví dụ: Ứng dụng ngân hàng hỗ trợ quét mã QR bằng camera.

Ưu điểm của phát triển gốc

Hiệu suất cao: Ứng dụng được tối ưu hóa tốt cho phần cứng.

Trải nghiệm người dùng mượt mà hơn nhờ tận dụng API gốc.

Khả năng tích hợp sâu: Hỗ trợ các tính năng độc quyền như Apple Pay hoặc Google Pay.

Dễ dàng mở rộng và cập nhật tính năng theo hệ điều hành.

Ví dụ: Ứng dụng Spotify tận dụng tính năng phát nhạc nền đặc thù của Android.

Nhược điểm của phát triển gốc

Chi phí cao: Cần đội ngũ riêng biệt cho mỗi nền tảng.

Thời gian phát triển dài hơn do không chia sẻ mã nguồn.

Khó bảo trì: Khi cập nhật, cần thay đổi từng phiên bản ứng dụng.

Đối mặt với thách thức về đồng bộ tính năng trên các nền tảng khác nhau.

Ví dụ: Một ứng dụng thương mại điện tử cần phát triển 2 bộ mã cho Android và iOS.

Phương pháp phát triển đa nền tảng (Cross-Platform)

Cho phép viết một mã nguồn duy nhất, chạy trên nhiều nền tảng.

Các công cụ phổ biến: Flutter, React Native, Xamarin.

Giảm thời gian và chi phí phát triển so với phương pháp gốc.

Hạn chế: Hiệu suất thấp hơn, không truy cập hết tính năng hệ điều hành.

Ví dụ: Ứng dụng Google Ads sử dụng Flutter để hỗ trợ cả Android và iOS.

Phát triển ứng dụng web di động (Mobile Web)

Ứng dụng chạy trên trình duyệt di động thay vì cài đặt trực tiếp.

Sử dụng các công nghệ như HTML5, CSS3, JavaScript.

Các ứng dụng web hiện đại (PWA) có thể hoạt động ngoại tuyến.

Ưu điểm: Không cần cài đặt, chi phí phát triển thấp.

Ví dụ: Twitter Lite sử dụng PWA cho trải nghiệm nhanh hơn trên di động.

Ưu điểm của ứng dụng web di động

Khả năng tương thích cao với nhiều thiết bị và trình duyệt.

Chi phí phát triển thấp hơn do không cần viết mã riêng cho mỗi nền tảng.

Dễ dàng cập nhật ứng dụng mà không cần qua chợ ứng dụng.

Không cần dung lượng lưu trữ lớn trên thiết bị.

Ví dụ: Google Maps Web vẫn cung cấp tính năng định vị và dẫn đường.

Nhược điểm của ứng dụng web di động

Hiệu năng kém hơn so với ứng dụng gốc.

Không truy cập được toàn bộ tính năng phần cứng như GPS, camera.

Trải nghiệm người dùng không mượt mà như ứng dụng gốc.

Yêu cầu kết nối mạng ổn định (trừ PWA).

Ví dụ: Ứng dụng web không thể hỗ trợ AR/VR như ứng dụng gốc.

Công cụ lập trình hiện đại

Figma: Thiết kế UI/UX trực tuyến với khả năng cộng tác.

Android Studio: IDE chính thức để phát triển ứng dụng Android.

Xcode: IDE chính thức để phát triển ứng dụng iOS.

Firebase: Cung cấp cơ sở dữ liệu, xác thực, phân tích và nhiều tính năng khác.

Ví dụ: Firebase hỗ trợ push notifications mà không cần backend riêng.

Flutter SDK và React Native

Flutter: SDK từ Google, sử dụng ngôn ngữ Dart, hỗ trợ đa nền tảng.

React Native: Framework của Facebook, sử dụng JavaScript.

Ưu điểm của Flutter: Giao diện đẹp, hiệu năng gần tương đương ứng dụng gốc.

Ưu điểm của React Native: Cộng đồng lớn, dễ học cho người quen JavaScript.

Ví dụ: Ứng dụng eBay Motors được phát triển bằng Flutter.

Giới thiệu về Android Studio

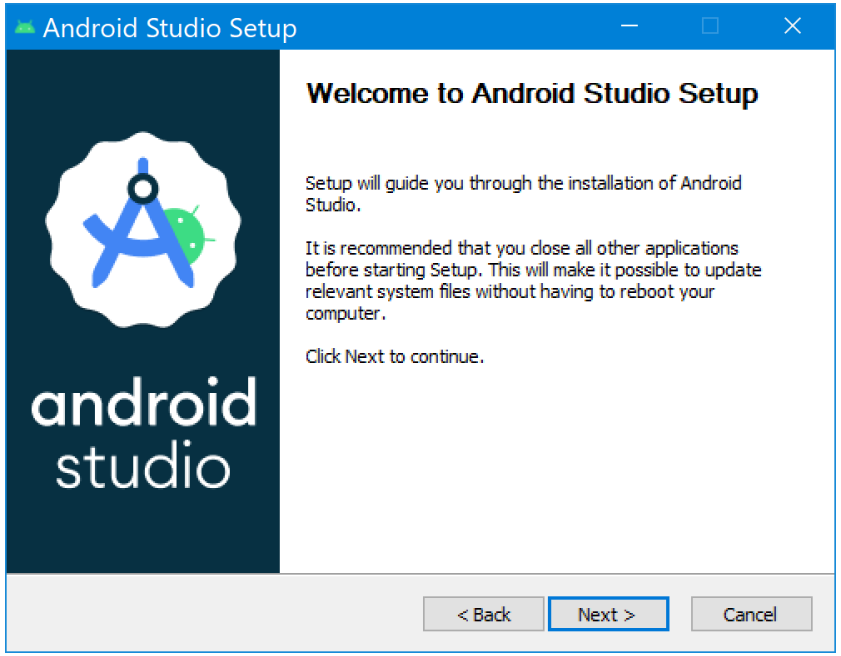
Android Studio là IDE chính thức của Google cho lập trình Android.

Hỗ trợ Kotlin, Java và C++.

Tích hợp công cụ giả lập (AVD) và phân tích mã.

Hỗ trợ xây dựng giao diện trực quan với Layout Editor.

Ví dụ: Tạo ứng dụng 'Hello World'.



Thiết bị ảo Android (AVD)

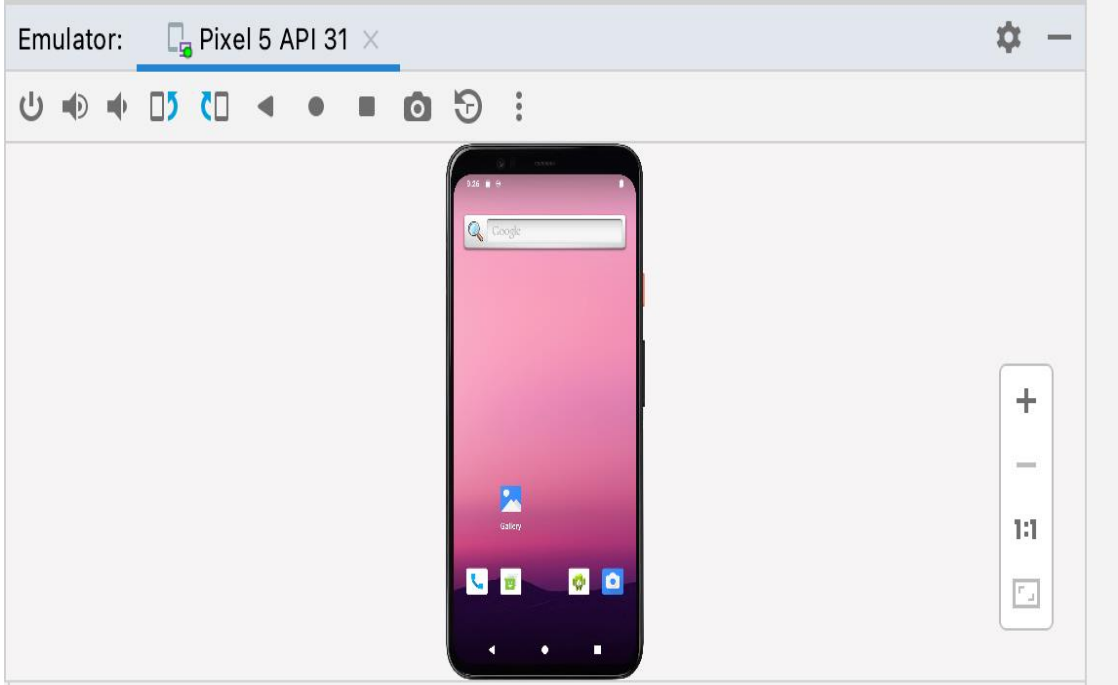
AVD mô phỏng các thiết bị thật để kiểm tra ứng dụng.

Tạo thiết bị ảo bằng công cụ Device Manager trong Android Studio.

Chọn phiên bản Android và cấu hình thiết bị.

Hỗ trợ kiểm tra trên nhiều kích thước màn hình và cấu hình phần cứng.

Ví dụ: Tạo thiết bị Pixel 5



Kết nối thiết bị thật để kiểm thử

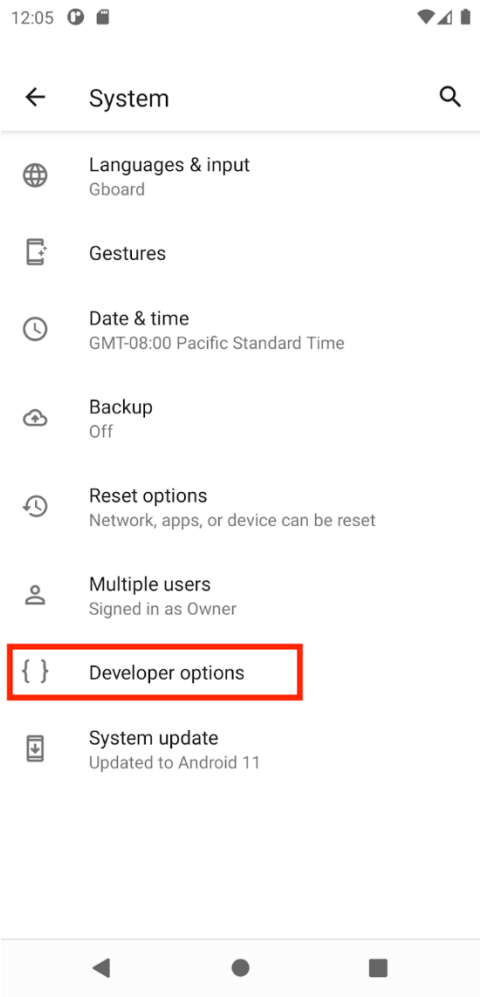
Bật chế độ nhà phát triển trên thiết bị Android.

Bật gỡ lỗi USB để kết nối với Android Studio.

Kết nối thiết bị với máy tính qua cáp USB hoặc Wi-Fi.

Chọn thiết bị trong Android Studio để chạy ứng dụng.

Ví dụ: Kiểm tra ứng dụng trên điện thoại Samsung Galaxy.



Giới thiệu về Kotlin

- Ngôn ngữ lập trình hiện đại, được Google khuyến nghị từ 2017.
- Tương tác hoàn hảo với Java, dễ dàng sử dụng thư viện Java.
- Tính năng: Ngắn gọn, an toàn, hỗ trợ lập trình hàm và đối tượng.
- Hỗ trợ đầy đủ các công cụ của Android Studio.
- Ví dụ: Kotlin giảm mã thừa với tính năng data class.

Tại sao chọn Kotlin?

- Cú pháp ngắn gọn hơn Java, giảm thiểu lỗi lập trình.
- An toàn hơn với null: Tránh lỗi NullPointerException.
- Tương thích ngược: Chạy song song với mã Java cũ.
- Tích hợp tốt với công cụ như Android Studio và IntelliJ IDEA.
- Ví dụ: Sử dụng 'when' thay cho 'switch' để cải thiện mã nguồn.

Ví dụ cơ bản với Kotlin

- Tạo biến: val name = "John", var age = 25.
- Vòng lặp: for (i in 1..10) { println(i) }.
- Kiểm tra điều kiện: if (x > y) { println("x lớn hơn y") }.
- Ví dụ: Tính tổng các số từ 1 đến n.

```kotlin
fun main() {
    println("Hello, Kotlin!")
}
```

Cách thiết lập môi trường Kotlin

- Cài đặt JDK để chạy Kotlin trên JVM.
- Sử dụng IntelliJ IDEA hoặc Android Studio.
- Cài đặt plugin Kotlin nếu chưa có sẵn.
- Tạo dự án mới và chọn ngôn ngữ Kotlin.

Ví dụ: Dự án Android mới sử dụng Jetpack Compose để phát triển UI.

Tính năng nổi bật của Kotlin

Data classes: Tự động tạo equals(), hashCode(), toString().

Extension functions: Thêm hàm cho lớp hiện tại mà không cần sửa mã gốc.

Coroutines: Xử lý công việc bất đồng bộ dễ dàng và hiệu quả.

Null safety: Loại bỏ lỗi NullPointerException.

Ví dụ: Dùng extension để thêm hàm capitalize cho String.

Lập trình với Kotlin: Các khái niệm cơ bản

Khai báo biến: val (bất biến), var (có thể thay đổi).

Kiểu dữ liệu: Int, String, Boolean, Double.

Câu lệnh điều kiện: if, when (thay cho switch).

Vòng lặp: for, while, do-while.

Ví dụ: Tính tổng các số chẵn từ 1 đến 100.

Lập trình với Kotlin: Các khái niệm cơ bản

Khai báo biến: val (bất biến), var (có thể thay đổi).

Kiểu dữ liệu: Int, String, Boolean, Double.

Câu lệnh điều kiện: if, when (thay cho switch).

Vòng lặp: for, while, do-while.

Ví dụ: Tính tổng các số chẵn từ 1 đến 100.

- Mutable (Changeable)

```
var score = 10
```

- Immutable (Unchangeable)

```
val name = "Jennifer"
```

Lập trình với Kotlin: Các Kiểu dữ liệu

Type	Bits	Notes
Long	64	From -2^{63} to $2^{63}-1$
Int	32	From -2^{31} to $2^{31}-1$
Short	16	From -32768 to 32767
Byte	8	From -128 to 127

Type	Bits	Notes
Double	64	16 - 17 significant digits
Float	32	6 - 7 significant digits
Char	16	16-bit Unicode character
Boolean	8	True or false. Operations include: || - lazy disjunction, && - lazy conjunction, ! - negation

Lập trình với Kotlin: Các Phép toán

- Các phép toán toán học
- Phép toán tăng giảm
- Các phép so sánh hơn
- Phép gán
- Phép toán so sánh bằng

```
+ - * / %        1 + 1      =>      2
++ --
                 53 - 3     =>      50
< <=  > >=
                 50 / 10    =>      5
=
                 9 % 3      =>      0
== !=
```

Lập trình với Kotlin: Các Phép toán

```
1+1
⇒ kotlin.Int = 2
```

```
1.0/2.0
⇒ kotlin.Double = 0.5
```

```
53-3
⇒ kotlin.Int = 50
```

```
2.0*3.5
⇒ kotlin.Double = 7.0
```

```
50/10
⇒ kotlin.Int = 5
```

Ví dụ: Câu lệnh điều kiện Kotlin

if (x > 0) { println("x là số dương") }.

when (x) { 1 -> println("x là 1") else -> println("Không phải 1") }.

Sử dụng biểu thức when để thay thế switch.

Kết hợp nhiều điều kiện trong when.

Ví dụ: Phân loại số chẵn và lẻ bằng when.

Xử lý null trong Kotlin

Kotlin an toàn với null: Các biến không thể là null trừ khi được chỉ định rõ.

Khai báo nullable: var name: String? = null.

Sử dụng toán tử ?. để kiểm tra null trước khi gọi hàm.

Toán tử Elvis (?:) cung cấp giá trị thay thế khi gặp null.

Ví dụ: val length = name?.length ?: 0.

Câu lệnh if trong Kotlin

Sử dụng if để kiểm tra điều kiện logic.

Cú pháp: if (condition) { ... } else { ... }.

if cũng có thể dùng như một biểu thức trả về giá trị.

Ví dụ:

```
val max = if (a > b) a else b
println("Giá trị lớn nhất: $max")
```

Ví dụ 1: Sử dụng if trong Kotlin

Kiểm tra số âm hay dương:

```
val number = -10
if (number > 0) {
    println("Số dương")
} else {
    println("Số âm")
}
```

Kết quả: In ra 'Số âm'.

Ứng dụng: Kiểm tra điều kiện đơn giản trong lập trình.

Ví dụ 2: Sử dụng if lồng nhau

Kiểm tra số âm, dương hay bằng 0:

```
val number = 0
if (number > 0) {
    println("Số dương")
} else if (number < 0) {
    println("Số âm")
} else {
    println("Bằng 0")
}
```

Kết quả: In ra 'Bằng 0'.

Câu lệnh when trong Kotlin

when là phiên bản nâng cao của switch trong Java.

Dùng để kiểm tra các điều kiện khác nhau.

Cú pháp:

```
when (variable) {
    value1 -> action1
    value2 -> action2
    else -> defaultAction
}
```

Ví dụ: Kiểm tra ngày trong tuần.

Ví dụ 1: Sử dụng when trong Kotlin

Kiểm tra ngày trong tuần:

```
val day = 3
val result = when (day) {
    1 -> "Thứ Hai"
    2 -> "Thứ Ba"
    3 -> "Thứ Tư"
    else -> "Không hợp lệ"
}
println(result)
```

Kết quả: In ra 'Thứ Tư'.

Ví dụ 2: Sử dụng when với phạm vi

Kiểm tra số thuộc phạm vi:

```
val number = 15
val result = when (number) {
    in 1..10 -> "1 đến 10"
    in 11..20 -> "11 đến 20"
    else -> "Ngoài phạm vi"
}
println(result)
```

Kết quả: In ra '11 đến 20'.

Ứng dụng: Phân loại giá trị theo khoảng.

Vòng lặp for trong Kotlin

for dùng để lặp qua một dãy số hoặc danh sách.

Cú pháp cơ bản:

```
for (i in 1..10) {
    println(i)
}
```

Cú pháp với step:

```
for (i in 1..10 step 2) {
    println(i)
}.
```

```
for (i in 1..5) print(i)
⇒ 12345

for (i in 5 downTo 1) print(i)
⇒ 54321

for (i in 3..6 step 2) print(i)
⇒ 35

for (i in 'd'..'g') print (i)
⇒ defg
```

Ví dụ 1: Tính tổng các số trong dãy

```
var sum = 0
for (i in 1..100) {
    sum += i
}
println("Tổng: $sum")
```

Kết quả: In ra tổng từ 1 đến 100.

Ứng dụng: Tính toán tổng giá trị dãy số.

Vòng lặp while trong Kotlin

while lặp lại khi điều kiện đúng.

Cú pháp:

```
while (condition) {
    // hành động
}
```

Sử dụng để kiểm tra điều kiện trước khi thực hiện.

Ví dụ: Tính tổng dãy số.

Ví dụ 1: Tính tổng dãy số với while

```
var sum = 0
var i = 1
while (i <= 10) {
    sum += i
    i++
}
println("Tổng: $sum")
```

Kết quả: In ra tổng từ 1 đến 10.

Ứng dụng: Tính toán giá trị khi điều kiện thay đổi.

Ví dụ 2: Lặp vô hạn với while

```
var count = 0
while (true) {
    count++
    if (count == 5) break
    println("Đếm: $count")
}
```

Kết quả: Dừng khi count = 5.

Ứng dụng: Kiểm soát vòng lặp với điều kiện động.

Hàm trong Kotlin

Hàm là khối mã thực hiện một nhiệm vụ cụ thể.

Khai báo hàm sử dụng từ khóa fun.

Cú pháp cơ bản:

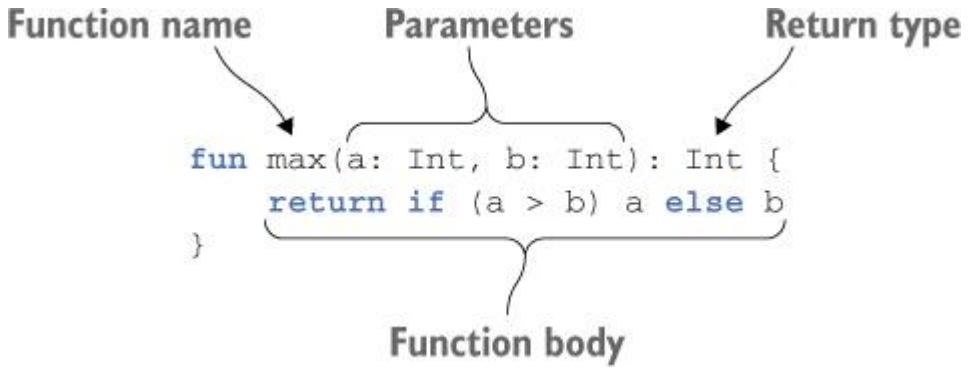
fun functionName(parameters): ReturnType {

// thân hàm

}

Ví dụ: Hàm max()

Ví dụ: Hàm tính tổng hai số.



Hàm tính tổng hai số

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
println(sum(3, 5))
```

Kết quả: In ra 8.

Ứng dụng: Tính toán với các giá trị đầu vào.

Tham số mặc định trong hàm

Có thể gán giá trị mặc định cho tham số.

Khi gọi hàm, có thể bỏ qua tham số có giá trị mặc định.

Cú pháp:

```
fun greet(name: String = "World") {
    println("Hello, $name!")
}
```

Ví dụ: greet() in ra 'Hello, World!'.

Tham số được đặt tên

Khi gọi hàm, có thể chỉ rõ tên tham số để tăng tính rõ ràng.

Không cần theo thứ tự khai báo nếu dùng tham số đặt tên.

Ví dụ:

```
fun greet(name: String, age: Int) {
    println("$name is $age years old")
}
greet(age = 25, name = "Alice")
```

Kết quả: In ra 'Alice is 25 years old'.

Hàm trả về giá trị

Hàm có thể trả về giá trị với từ khóa return.

Cần chỉ rõ kiểu trả về sau dấu hai chấm (:) trong khai báo hàm.

Cú pháp:

```
fun multiply(a: Int, b: Int): Int {
    return a * b
}
```

Ví dụ: multiply(4, 5) trả về 20.

Hàm không trả về giá trị

Hàm không trả về giá trị sử dụng kiểu Unit (có thể bỏ qua).

Cú pháp:

```
fun printMessage(message: String) {
    println(message)
}
```

Ví dụ: printMessage("Hello, Kotlin!") in ra 'Hello, Kotlin!'.

Ứng dụng: Các hàm thực hiện hành động như in thông tin.

Hàm mở rộng (Extension Functions)

Thêm chức năng mới vào lớp mà không sửa mã gốc.

Cú pháp:

```
fun ClassName.methodName() { ... }
```

Ví dụ:

```
fun String.capitalizeFirst(): String {
    return this.replaceFirstChar { it.uppercase() }
}
println("kotlin".capitalizeFirst())
```

Kết quả: In ra 'Kotlin'.

Tổng kết về hàm trong Kotlin

Hàm là thành phần quan trọng giúp tổ chức mã nguồn.

Hỗ trợ các tính năng mạnh như tham số mặc định, hàm lambda.

Cú pháp đơn giản và dễ đọc.

Tăng khả năng tái sử dụng và bảo trì mã nguồn.

Thực hành: Viết các hàm xử lý dữ liệu cơ bản trong ứng dụng.

Câu hỏi thảo luận

Lập trình di động có ảnh hưởng như thế nào đến cuộc sống hàng ngày?

Những xu hướng mới trong lập trình di động hiện nay là gì?

So sánh giữa Kotlin và Java trong lập trình Android. Ngôn ngữ nào phù hợp hơn và vì sao?

Những thách thức lớn nhất khi học lập trình di động là gì? Làm thế nào để vượt qua?

Đề xuất một ý tưởng ứng dụng di động hữu ích. Ứng dụng đó giải quyết vấn đề gì?

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024